

- The assignment is due at Gradescope on 4/4/25.
- A LaTeX template will be provided for each homework. You are strongly encouraged to type your homework into this template using $\text{\LaTeX}$. If you are writing by hand, please fill in the solutions in this template, inserting additional sheets as necessary. This will help facilitate the grading.
- You are permitted to discuss the problems with up to 2 other students in the class (per problem); however, *you must write up your own solutions, in your own words*. Do not submit anything you cannot explain. If you do collaborate with any of the other students on any problem, please list all your collaborators in the appropriate spaces.
- Similarly, please list any other source you have used for each problem, including other textbooks or websites.
- *Show your work*. Answers without justification will be given little credit.
- Your homework is *resubmittable*. Please refer to the course syllabus on Canvas for a more detailed description of this. For any problem that you have not changed from your last submission, please make sure to indicate this in your submission to help our graders grade faster.
- Each student can request a total of up to 5 days of extensions on all homework - either for original submission or resubmission. Up to two days can be requested for a homework. If you would like to request an extension, please fill out the [Homework extension form](#). Extensions do not carry any late penalty. If there is a life emergency that might prevent you from submitting on time not covered by this policy, email the Head TA.

PROBLEM 1 Consider the recursive version of the gcd algorithm given in class and in the DPV textbook.

1. How much space, i.e., memory allocation, does this algorithm require as stated?
2. Write pseudocode for an iterative version of the algorithm that reduces the space requirement to $O(n)$ bits.

Collaborators:

Solution: Your solution here.

PROBLEM 2 Let F_i be the i th Fibonacci number. Consider the execution of the recursive gcd algorithm on inputs (a, b) with $a > b \geq 0$. Show that if $b \leq F_k$ for some k , then the algorithm makes at most k recursive calls.

Collaborators:

Solution: Your solution here.

PROBLEM 3 Wilson's theorem says that a number N is prime if and only if

$$(N - 1)! \equiv -1 \pmod{N}.$$

In this problem, we will walk through writing a formal proof for this theorem.

(a) We can prove Wilson's Theorem by proving three lemmas:

Lemma 1 If p is prime, only 1 and $p - 1$ are their own inverses modulo p .

Lemma 2 If p is prime, $(p - 1)! \equiv -1 \pmod{p}$.

Lemma 3 ?????

What is the third lemma that needs to be proven for a complete proof of Wilson's Theorem?

(b) Provide a proof of Lemma 1. You may use the fact that if p is prime, then we know every number $1 \leq x < p$ is invertible modulo p .

(c) Provide a proof of Lemma 2 by pairing up multiplicative inverses. You may use the fact that every number $1 \leq x < p$ has a **unique** inverse modulo p .

(d) Provide a proof of Lemma 3. (Hint: Note that this can only happen if:

$$\gcd(N, (N - 1)!) = 1.$$

)

(e) We've just shown that Wilson's theorem is an if-and-only-if condition for primality. Why can't we base a primality test algorithm on Wilson's Theorem?

Collaborators:

Solution: [Your solution here.](#)

PROBLEM 4 (PROGRAMMING: PROBABILISTIC TESTING AND PRIMALITY) Follow this [Google Colab template link](#) to implement Fermat primality testing. **Do not attempt to edit the linked file. Create your own copy instead by clicking File → Save a copy in Drive.** When you have completed it, download it as a Jupyter Notebook and submit it to the appropriate Gradescope assignment.

Collaborators:

Solution: Your solution